

ASSIGNMENT-12.5

2303A51042

BATCH – 29

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
- Instructions:
 - o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.
 - o Ask AI to include time complexity and space complexity in the function docstring.
 - o Verify the generated code with test cases.
- Expected Output:
 - o A functional Python script implementing Merge Sort with proper documentation.

CODE:

Users > zohaib > Documents > AI ASSISTANT > AI ASSISTANT LAB.py > 12.5(T1).py > merge

```
1 def merge_sort(arr):
2     """
3     Sort a list using the Merge Sort algorithm.
4
5     Merge Sort works by dividing the list into two halves,
6     recursively sorting each half, and then merging them
7     back together in sorted order.
8
9     Time Complexity:
10    | Best Case: O(n log n)
11    | Average Case: O(n log n)
12    | Worst Case: O(n log n)
13
14    Space Complexity:
15    | O(n) additional space is required for merging lists.
16
17    Args:
18    | arr (list): List of numbers to be sorted.
19
20    Returns:
21    | list: Sorted list in ascending order.
22    """
23
24    if len(arr) <= 1:
25        return arr
26
27    mid = len(arr) // 2
28    left_half = merge_sort(arr[:mid])
29    right_half = merge_sort(arr[mid:])
30
31    return merge(left_half, right_half)
32
33 def merge(left, right):
34     """Merge two sorted lists into one sorted list."""
35     sorted_list = []
36     i = j = 0
37
38     while i < len(left) and j < len(right):
39         if left[i] <= right[j]:
40             sorted_list.append(left[i])
41             i += 1
42         else:
43             sorted_list.append(right[j])
44             j += 1
45
46     sorted_list.extend(left[i:])
47     sorted_list.extend(right[j:])
48
49     return sorted_list
50
51 if __name__ == "__main__":
52     test1 = [5, 3, 8, 4, 2]
53     test2 = [10, 9, 8, 7, 6]
54     test3 = [1]
55     test4 = []
56     test5 = [4, 1, 3, 2, 5]
57
58     print("Original:", test1)
59     print("Sorted:", merge_sort(test1))
60
61     print("Original:", test2)
62     print("Sorted:", merge_sort(test2))
63
64     print("Original:", test3)
65     print("Sorted:", merge_sort(test3))
66
67     print("Original:", test4)
68     print("Sorted:", merge_sort(test4))
69
70     print("Original:", test5)
71     print("Sorted:", merge_sort(test5))
```

OUTPUT:

```
/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T1).py"
● zohaib@MacBook-2 ~ % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T1).py"
Original: [5, 3, 8, 4, 2]
Sorted: [2, 3, 4, 5, 8]
Original: [10, 9, 8, 7, 6]
Sorted: [6, 7, 8, 9, 10]
Original: [1]
Sorted: [1]
Original: []
Sorted: []
Original: [4, 1, 3, 2, 5]
Sorted: [1, 2, 3, 4, 5]
○ zohaib@MacBook-2 ~ %
```

OBSERVATION:

The Merge Sort algorithm was implemented using the divide-and-conquer technique. The input list was recursively divided into smaller sublists and merged in sorted order. Test cases confirmed correct sorting of lists with different sizes. The algorithm operates with **$O(n \log n)$** time complexity and **$O(n)$** space complexity.

Task Description #2 (Searching – Binary Search with AI

Optimization)

- **Task:** Use AI to create a binary search function that finds a target element in a sorted list.

- **Instructions:**

- o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.

- o Include docstrings explaining best, average, and worst-case complexities.

- o Test with various inputs.

- **Expected Output:**

- o Python code implementing binary search with AI-generated comments and docstrings.

CODE:

```

12.5(T1).py 12.5(T2).py X 12.5(T3).py 12.5(T4).py 12.5(T5).py
Users > zohaib > Documents > AI ASSISTANT > AI ASSISTANT LAB.py > 12.5(T2).py > ...
1 def binary_search(arr, target):
2     """
3     Perform Binary Search on a sorted list to find the target element.
4
5     Binary Search works by repeatedly dividing the search interval
6     in half. It compares the middle element with the target and
7     eliminates half of the remaining elements.
8
9     Time Complexity:
10    | Best Case: O(1) | Target found at middle immediately
11    | Average Case: O(log n)
12    | Worst Case: O(log n)
13
14    Space Complexity:
15    | O(1) | Iterative implementation uses constant space.
16
17    Args:
18    | arr (list): Sorted list of elements
19    | target (int): Element to search for
20
21    Returns:
22    | int: Index of target if found, otherwise -1
23    """
24
25    left = 0
26    right = len(arr) - 1
27
28    while left <= right:
29        mid = (left + right) // 2
30
31        if arr[mid] == target:
32            return mid
33        elif arr[mid] < target:
34            left = mid + 1
35        else:
36            right = mid - 1
37
38    return -1
39
40
41 if __name__ == "__main__":
42     numbers = [2, 4, 6, 8, 10, 12, 14, 16]
43
44     print("Array:", numbers)
45
46     print("Search 10 → Index:", binary_search(numbers, 10))
47     print("Search 2 → Index:", binary_search(numbers, 2))
48     print("Search 16 → Index:", binary_search(numbers, 16))
49     print("Search 7 → Index:", binary_search(numbers, 7))

```

OUTPUT:

```

/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T2).py"
● zohaib@MacBook-2 ~ % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T2).py"
Array: [2, 4, 6, 8, 10, 12, 14, 16]
Search 10 → Index: 4
Search 2 → Index: 0
Search 16 → Index: 7
Search 7 → Index: -1
○ zohaib@MacBook-2 ~ %

```

OBSERVATION:

Binary Search was implemented to efficiently locate a target element in a sorted list. The algorithm repeatedly divides the search space in half until the element is found or the search space becomes empty. The implementation successfully returned the correct index or **-1** when the element was not present. The algorithm runs in **$O(\log n)$** time complexity.

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

CODE:

```
12.5(T1).py 12.5(T2).py 12.5(T3).py 12.5(T4).py 12.5(T5).py 12.5(T6).py 12.5(T7).py 12.5(T8).py
Users > zohaib > Documents > AI ASSISTANT > AI ASSISTANT LAB.py > 12.5(T3).py > ...
1 appointments = [
2     {"id": 101, "patient": "Ram", "doctor": "Dr. Sharma", "time": "09:30", "fee": 500},
3     {"id": 102, "patient": "Sita", "doctor": "Dr. Mehta", "time": "10:15", "fee": 700},
4     {"id": 103, "patient": "Ravi", "doctor": "Dr. Rao", "time": "08:45", "fee": 400},
5     {"id": 104, "patient": "Anita", "doctor": "Dr. Khan", "time": "11:00", "fee": 650}
6 ]
7 def binary_search(arr, target_id):
8     left = 0
9     right = len(arr) - 1
10
11     while left <= right:
12         mid = (left + right) // 2
13
14         if arr[mid]["id"] == target_id:
15             return arr[mid]
16         elif arr[mid]["id"] < target_id:
17             left = mid + 1
18         else:
19             right = mid - 1
20
21     return None
22
23
24 def merge_sort(arr, key):
25     if len(arr) <= 1:
26         return arr
27
28     mid = len(arr) // 2
29     left = merge_sort(arr[:mid], key)
30     right = merge_sort(arr[mid:], key)
31
32     return merge(left, right, key)
33
34
35 def merge(left, right, key):
36     result = []
37     i = j = 0
38
39     while i < len(left) and j < len(right):
40         if left[i][key] <= right[j][key]:
41             result.append(left[i])
42             i += 1
43         else:
44             result.append(right[j])
45             j += 1
46
47     result.extend(left[i:])
48     result.extend(right[j:])
49     return result
50
51
52 # Sort appointments by ID before searching
53 appointments = merge_sort(appointments, "id")
54
55 print("Search Appointment ID 103:")
56 print(binary_search(appointments, 103))
57
58
59 print("\nSort by Consultation Fee:")
60 sorted_fee = merge_sort(appointments, "fee")
61 for a in sorted_fee:
62     print(a)
63
64
65 print("\nSort by Appointment Time:")
66 sorted_time = merge_sort(appointments, "time")
67 for a in sorted_time:
68     print(a)
```

OUTPUT:

```

/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T3).py"
zohaib@MacBook-2 ~ % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T3).py"
Search Appointment ID 103:
{'id': 103, 'patient': 'Ravi', 'doctor': 'Dr. Rao', 'time': '08:45', 'fee': 400}

Sort by Consultation Fee:
{'id': 103, 'patient': 'Ravi', 'doctor': 'Dr. Rao', 'time': '08:45', 'fee': 400}
{'id': 101, 'patient': 'Ram', 'doctor': 'Dr. Sharma', 'time': '09:30', 'fee': 500}
{'id': 104, 'patient': 'Anita', 'doctor': 'Dr. Khan', 'time': '11:00', 'fee': 650}
{'id': 102, 'patient': 'Sita', 'doctor': 'Dr. Mehta', 'time': '10:15', 'fee': 700}

Sort by Appointment Time:
{'id': 103, 'patient': 'Ravi', 'doctor': 'Dr. Rao', 'time': '08:45', 'fee': 400}
{'id': 101, 'patient': 'Ram', 'doctor': 'Dr. Sharma', 'time': '09:30', 'fee': 500}
{'id': 102, 'patient': 'Sita', 'doctor': 'Dr. Mehta', 'time': '10:15', 'fee': 700}
{'id': 104, 'patient': 'Anita', 'doctor': 'Dr. Khan', 'time': '11:00', 'fee': 650}
zohaib@MacBook-2 ~ %

```

OBSERVATION:

AI-assisted analysis recommended Binary Search for appointment ID lookup and Merge Sort for organizing appointment records. The system successfully searched appointments and sorted them by consultation fee or appointment time. This improved efficiency and ensured structured data management.

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

CODE:

```
12.5(T1).py 12.5(T2).py 12.5(T3).py 12.5(T4).py 12.5(T5).py 12.5(T6).py 12.5(T7).py
Users > zohaib > Documents > AI ASSISTANT > AI ASSISTANT LAB.py > 12.5(T4).py > ...
1 # Railway ticket booking records
2 tickets = [
3     {"id": 201, "passenger": "Ram", "train": 12345, "seat": 21, "date": "2024-06-10"},
4     {"id": 202, "passenger": "Sita", "train": 54321, "seat": 12, "date": "2024-06-08"},
5     {"id": 203, "passenger": "Ravi", "train": 67890, "seat": 30, "date": "2024-06-12"},
6     {"id": 204, "passenger": "Anita", "train": 98765, "seat": 5, "date": "2024-06-09"}
7 ]
8 # ----- Binary Search for Ticket ID -----
9 def binary_search(arr, target_id):
10     left = 0
11     right = len(arr) - 1
12
13     while left <= right:
14         mid = (left + right) // 2
15
16         if arr[mid]["id"] == target_id:
17             return arr[mid]
18         elif arr[mid]["id"] < target_id:
19             left = mid + 1
20         else:
21             right = mid - 1
22
23     return None
24 # ----- Merge Sort -----
25 def merge_sort(arr, key):
26     if len(arr) <= 1:
27         return arr
28
29     mid = len(arr) // 2
30     left = merge_sort(arr[:mid], key)
31     right = merge_sort(arr[mid:], key)
32
33     return merge(left, right, key)
34
35 def merge(left, right, key):
36     result = []
37     i = j = 0
38
39     while i < len(left) and j < len(right):
40         if left[i][key] <= right[j][key]:
41             result.append(left[i])
42             i += 1
43         else:
44             result.append(right[j])
45             j += 1
46
47     result.extend(left[i:])
48     result.extend(right[j:])
49     return result
50
51 # ----- Demonstration -----
52
53 # Sort tickets by ID before searching
54 tickets = merge_sort(tickets, "id")
55
56 print("Search Ticket ID 203:")
57 print(binary_search(tickets, 203))
58
59 print("\nSort by Travel Date:")
60 sorted_date = merge_sort(tickets, "date")
61 for t in sorted_date:
62     print(t)
63
64 print("\nSort by Seat Number:")
65 sorted_seat = merge_sort(tickets, "seat")
66 for t in sorted_seat:
67     print(t)
```

OUTPUT:

```
/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T4).py"
zohaib@MacBook-2 ~ % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T4).py"
Search Ticket ID 203:
{'id': 203, 'passenger': 'Ravi', 'train': 67890, 'seat': 30, 'date': '2024-06-12'}

Sort by Travel Date:
{'id': 202, 'passenger': 'Sita', 'train': 54321, 'seat': 12, 'date': '2024-06-08'}
{'id': 204, 'passenger': 'Anita', 'train': 98765, 'seat': 5, 'date': '2024-06-09'}
{'id': 201, 'passenger': 'Ram', 'train': 12345, 'seat': 21, 'date': '2024-06-10'}
{'id': 203, 'passenger': 'Ravi', 'train': 67890, 'seat': 30, 'date': '2024-06-12'}

Sort by Seat Number:
{'id': 204, 'passenger': 'Anita', 'train': 98765, 'seat': 5, 'date': '2024-06-09'}
{'id': 202, 'passenger': 'Sita', 'train': 54321, 'seat': 12, 'date': '2024-06-08'}
{'id': 201, 'passenger': 'Ram', 'train': 12345, 'seat': 21, 'date': '2024-06-10'}
{'id': 203, 'passenger': 'Ravi', 'train': 67890, 'seat': 30, 'date': '2024-06-12'}
zohaib@MacBook-2 ~ %
```


OBSERVATION:

Efficient algorithms were applied to manage railway booking records. Binary Search enabled quick retrieval of ticket details using ticket ID, while Merge Sort arranged bookings by travel date or seat number. The implementation demonstrated accurate searching and stable sorting.

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

CODE:

```

1 allocations = [
2     {"id":101, "room":205, "floor":2, "date":"2024-01-10"},
3     {"id":102, "room":101, "floor":1, "date":"2024-01-08"},
4     {"id":103, "room":310, "floor":3, "date":"2024-01-12"},
5     {"id":104, "room":150, "floor":1, "date":"2024-01-09"}
6 ]
7
8 def binary_search(arr, target):
9     left, right = 0, len(arr)-1
10    while left <= right:
11        mid = (left+right)//2
12        if arr[mid]["id"] == target:
13            return arr[mid]
14        elif arr[mid]["id"] < target:
15            left = mid+1
16        else:
17            right = mid-1
18    return None
19
20 def merge_sort(arr, key):
21     if len(arr) <= 1:
22         return arr
23     mid = len(arr)//2
24     left = merge_sort(arr[:mid], key)
25     right = merge_sort(arr[mid:], key)
26     return merge(left, right, key)
27
28 def merge(left, right, key):
29     result=[]
30     i=j=0
31     while i<len(left) and j<len(right):
32         if left[i][key] <= right[j][key]:
33             result.append(left[i]); i+=1
34         else:
35             result.append(right[j]); j+=1
36     result.extend(left[i:])
37     result.extend(right[j:])
38     return result
39
40 allocations = merge_sort(allocations,"id")
41
42 print("Search Student ID:", binary_search(allocations,103))
43
44 print("\nSort by Room Number:")
45 for a in merge_sort(allocations,"room"):
46     print(a)

```

OUTPUT :

```

/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T5).py"
● zohaib@MacBook-2 ~ % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T5).py"
Search Student ID: {'id': 103, 'room': 310, 'floor': 3, 'date': '2024-01-12'}

Sort by Room Number:
{'id': 102, 'room': 101, 'floor': 1, 'date': '2024-01-08'}
{'id': 104, 'room': 150, 'floor': 1, 'date': '2024-01-09'}
{'id': 101, 'room': 205, 'floor': 2, 'date': '2024-01-10'}
{'id': 103, 'room': 310, 'floor': 3, 'date': '2024-01-12'}
○ zohaib@MacBook-2 ~ %

```

OBSERVATION :

The hostel allocation system used Binary Search to quickly locate student records by student ID and Merge Sort to organize allocations by room number or date. The approach improved search efficiency and allowed structured management of room allocation data.

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions.

CODE;

Users > zohaib > Documents > AI ASSISTANT > AI ASSISTANT LAB.py > 12.5(T6).py > merge

```
1  movies = [
2      {"id":1,"title":"Inception","genre":"Sci-Fi","rating":8.8,"year":2010},
3      {"id":2,"title":"Avatar","genre":"Action","rating":7.8,"year":2009},
4      {"id":3,"title":"Interstellar","genre":"Sci-Fi","rating":8.6,"year":2014},
5      {"id":4,"title":"Titanic","genre":"Drama","rating":7.9,"year":1997}
6  ]
7
8  def binary_search(arr,target):
9      l,r=0,len(arr)-1
10     while l<=r:
11         m=(l+r)//2
12         if arr[m]["id"]==target:
13             return arr[m]
14         elif arr[m]["id"]<target:
15             l=m+1
16         else:
17             r=m-1
18     return None
19
20 def merge_sort(arr,key):
21     if len(arr)<=1:
22         return arr
23     mid=len(arr)//2
24     left=merge_sort(arr[:mid],key)
25     right=merge_sort(arr[mid:],key)
26     return merge(left,right,key)
27
28 def merge(left,right,key):
29     result=[]
30     i=j=0
31     while i<len(left) and j<len(right):
32         if left[i][key] <= right[j][key]:
33             result.append(left[i]); i+=1
34         else:
35             result.append(right[j]); j+=1
36     result.extend(left[i:])
37     result.extend(right[j:])
38     return result
39
40 movies = merge_sort(movies,"id")
41
42 print("Search Movie ID:", binary_search(movies,3))
43
44 print("\nSort by Rating:")
45 for m in merge_sort(movies,"rating"):
46     print(m)
```

OUTPUT:

```
/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T6).py"
zohaib@MacBook-2 ~ % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T6).py"
Search Movie ID: {'id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'year': 2014}

Sort by Rating:
{'id': 2, 'title': 'Avatar', 'genre': 'Action', 'rating': 7.8, 'year': 2009}
{'id': 4, 'title': 'Titanic', 'genre': 'Drama', 'rating': 7.9, 'year': 1997}
{'id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'year': 2014}
{'id': 1, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'year': 2010}
zohaib@MacBook-2 ~ %
```

OBSERVATION:

Binary Search was used to efficiently search movies by movie ID, while Merge Sort organized movies based on rating or release year. The implementation demonstrated efficient handling of movie data and accurate retrieval of information.

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

CODE:

```

Users > zohaib > Documents > AI ASSISTANT > AI ASSISTANT LAB.py > 12.5(T7).py
1  crops = [
2      {"id":1,"name":"Rice","moisture":70,"temp":30,"yield":4.5},
3      {"id":2,"name":"Wheat","moisture":50,"temp":25,"yield":3.2},
4      {"id":3,"name":"Corn","moisture":60,"temp":28,"yield":4.0},
5      {"id":4,"name":"Barley","moisture":45,"temp":22,"yield":2.8}
6  ]
7
8  def binary_search(arr,target):
9      l,r=0,len(arr)-1
10     while l<=r:
11         m=(l+r)//2
12         if arr[m]["id"]==target:
13             return arr[m]
14         elif arr[m]["id"]<target:
15             l=m+1
16         else:
17             r=m-1
18     return None
19
20 def merge_sort(arr,key):
21     if len(arr)<=1:
22         return arr
23     mid=len(arr)//2
24     left=merge_sort(arr[:mid],key)
25     right=merge_sort(arr[mid:],key)
26     return merge(left,right,key)
27
28 def merge(left,right,key):
29     result=[]
30     i=j=0
31     while i<len(left) and j<len(right):
32         if left[i][key] <= right[j][key]:
33             result.append(left[i]); i+=1
34         else:
35             result.append(right[j]); j+=1
36     result.extend(left[i:])
37     result.extend(right[j:])
38     return result
39
40 crops = merge_sort(crops,"id")
41
42 print("Search Crop ID:", binary_search(crops,2))
43
44 print("\nSort by Moisture Level:")
45 for c in merge_sort(crops,"moisture"):
46     print(c)

```

OUTPUT:

```

/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T7).py"
zohaib@MacBook-2 ~ % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T7).py"
Search Crop ID: {'id': 2, 'name': 'Wheat', 'moisture': 50, 'temp': 25, 'yield': 3.2}

Sort by Moisture Level:
{'id': 4, 'name': 'Barley', 'moisture': 45, 'temp': 22, 'yield': 2.8}
{'id': 2, 'name': 'Wheat', 'moisture': 50, 'temp': 25, 'yield': 3.2}
{'id': 3, 'name': 'Corn', 'moisture': 60, 'temp': 28, 'yield': 4.0}
{'id': 1, 'name': 'Rice', 'moisture': 70, 'temp': 30, 'yield': 4.5}
zohaib@MacBook-2 ~ %

```

OBSERVATION:

Crop data was managed using Binary Search for fast retrieval of crop details by crop ID. Merge Sort was applied to arrange crops according to soil moisture level or yield estimate. This helped farmers analyze crop data efficiently.

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.
- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

CODE:

```

zohaib / Documents / AI ASSISTANT / AI ASSISTANT LAB.py / 12.5(T8).py / ...
flights = [
    {"id":101,"airline":"Indigo","depart":"09:30","arrive":"11:00","status":"On Time"},
    {"id":102,"airline":"Air India","depart":"08:45","arrive":"10:30","status":"Delayed"},
    {"id":103,"airline":"SpiceJet","depart":"12:15","arrive":"14:00","status":"On Time"},
    {"id":104,"airline":"Vistara","depart":"07:50","arrive":"09:20","status":"On Time"}
]

def binary_search(arr,target):
    l,r=0,len(arr)-1
    while l<=r:
        m=(l+r)//2
        if arr[m]["id"]==target:
            return arr[m]
        elif arr[m]["id"]<target:
            l=m+1
        else:
            r=m-1
    return None

def merge_sort(arr,key):
    if len(arr)<=1:
        return arr
    mid=len(arr)//2
    left=merge_sort(arr[:mid],key)
    right=merge_sort(arr[mid:],key)
    return merge(left,right,key)

def merge(left,right,key):
    result=[]
    i=j=0
    while i<len(left) and j<len(right):
        if left[i][key] <= right[j][key]:
            result.append(left[i]); i+=1
        else:
            result.append(right[j]); j+=1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

flights = merge_sort(flights,"id")

print("Search Flight ID:", binary_search(flights,102))

print("\nSort by Departure Time:")
for f in merge_sort(flights,"depart"):
    print(f)

```

OUTPUT:

```

/opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T8).py"
zohaib@MacBook-2 ~ % /opt/homebrew/bin/python3 "/Users/zohaib/Documents/AI ASSISTANT/AI ASSISTANT LAB.py/12.5(T8).py"
Search Flight ID: {'id': 102, 'airline': 'Air India', 'depart': '08:45', 'arrive': '10:30', 'status': 'Delayed'}

Sort by Departure Time:
{'id': 104, 'airline': 'Vistara', 'depart': '07:50', 'arrive': '09:20', 'status': 'On Time'}
{'id': 102, 'airline': 'Air India', 'depart': '08:45', 'arrive': '10:30', 'status': 'Delayed'}
{'id': 101, 'airline': 'Indigo', 'depart': '09:30', 'arrive': '11:00', 'status': 'On Time'}
{'id': 103, 'airline': 'SpiceJet', 'depart': '12:15', 'arrive': '14:00', 'status': 'On Time'}
zohaib@MacBook-2 ~ %

```

OBSERVATION:

The flight management system applied Binary Search to quickly locate flight records using flight ID. Merge Sort was used to organize flights based on departure or arrival time. The system successfully handled flight data with efficient searching and sorting operations.